



Lyon 1

Mesh and Computational Geometry

Raphaëlle Chaîne
Université Claude Bernard Lyon 1

M2 ID3D
Image, Développement
et Technologie 3D
et 3A Centrale

2025-26



1

Problem of a triangulating a surface passing through points

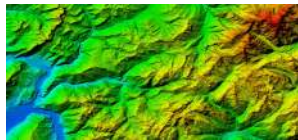
- 2D Case : points belonging to a plane
- Ideas for constructing a mesh from these data?
 - Connect the points together avoiding crossings to produce non overlapping triangles

85

85

Problem of a triangulating a surface passing through points

- Case of points belonging to a plane
- Case of a digital terrain model
 - Data can be parameterized as a height function with respect to a reference plane
 - 2D $\frac{1}{2}$ dimension
- Ideas for reconstructing from these data?



86

86

Triangulation of a terrain

- Amounts to a 2D Problem
 - Work on the projection of the points on the reference plane
 - Then move the vertices upward to their initial altitude

87

87

2D Triangulation

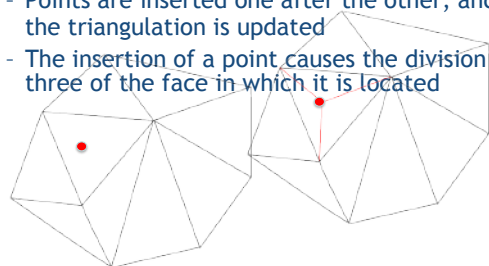
- Naive incremental construction
 - Add the points one after the other

88

88

2D Triangulation

- Naive incremental construction
 - Triangulation of 3 points: triangle oriented in the trigonometric direction
 - Points are inserted one after the other, and the triangulation is updated
 - The insertion of a point causes the division in three of the face in which it is located



89

89

2D Triangulation

- If the point to be inserted is outside every face:
 - The insertion of the point outside the convex hull creates new triangles: One for each boundary edge that is "visible by the point"
- The triangulation remains convex after each insertion

90

90

2D Triangulation

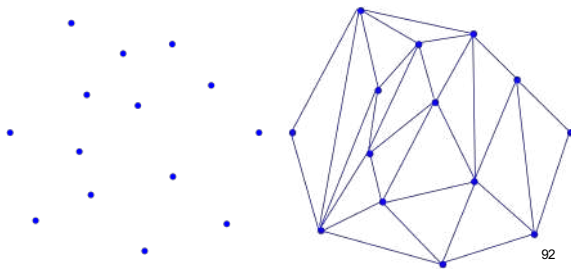
- Visibility test
 - Boundary edges are oriented (clockwise)
 - The oriented edge AB is visible by P if the triangle ABP is oriented counter-clockwise
 - Consider the sign of $(AB \times AP) \cdot k$
 - $A(x_A, y_A, 0), B(x_B, y_B, 0), P(x, y, 0), k(0, 0, 1)$
- Reminder :
 - edge of the convex envelope = pair (index of an infinite face + local index of the infinite vertex in that face)

91

91

2D Triangulation

- Naive incremental construction
 - Result clearly depending on the order in which the points are inserted



92

92

2D Triangulation

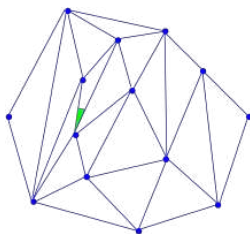
- Quality of a triangle
 - In most applications a "good triangle" is as equilateral as possible
 - Aspect ratio of a triangle
 - Inscribed circle radius / circumscribed circle radius
 - Minimum edge length / circumscribed circle radius
 - $\sin(\text{smallest angle})$
 - We would like triangles with aspect ratio as large as possible

93

93

Quality of a 2D triangulation

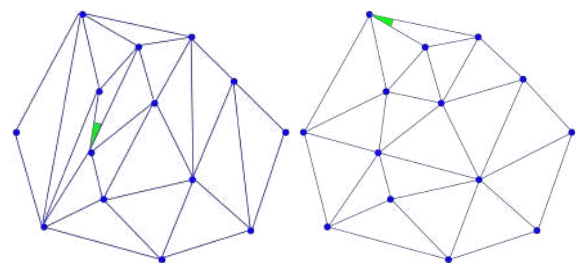
- Each triangulation is characterized by a smallest angle
- From all possible triangulations, choose one that maximizes the smallest angle



94

94

Quality of a 2D triangulation

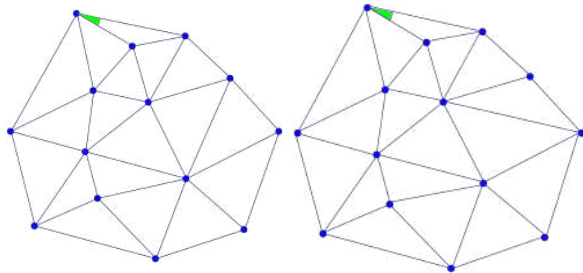


Optimal triangulation

95

95

Quality of a 2D triangulation



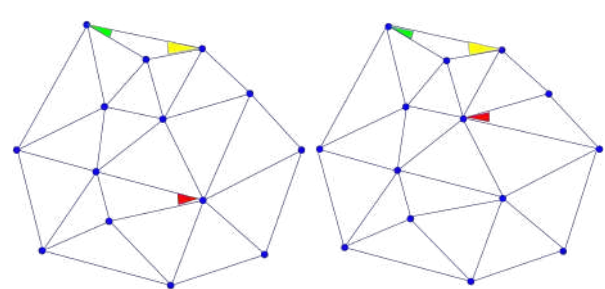
Optimal triangulation

What about this triangulation?

96

96

Quality of a 2D triangulation



Optimal triangulation

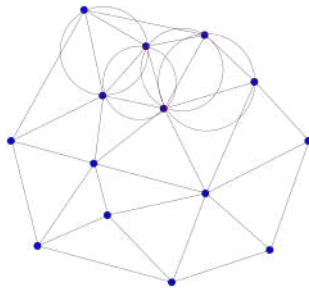
Triangulation not optimal in the lexicographic order of the smallest angles

97

97

Delaunay Triangulation

- Triangulation with triangles having an empty circumscribed circle



98

98

Theorem

- The triangulation that maximizes the smallest angles is the Delaunay triangulation

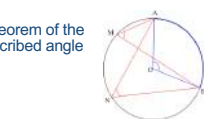
99

99

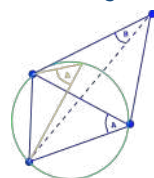
Proof of the equivalence

- Delaunay $\Leftrightarrow$ Maximizes the smallest angles
 - True for 4 points :
 - Delaunay ■ Maximizes the smallest angles :
 - Proof : Regardless of where the smallest angle is in the Delaunay triangulation, the alternative triangulation always contains smaller angles. Therefore, the smallest angle of the Delaunay triangulation is larger than the smallest angle of the alternative triangulation.

Theorem of the inscribed angle



$$(\vec{OA}, \vec{OB}) \equiv 2(\vec{MA}, \vec{MB}) \pmod{2\pi}$$



100

100

Proof of the equivalence

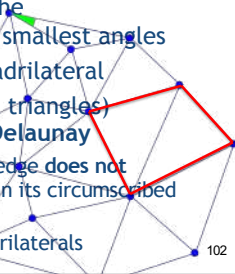
- Delaunay $\Leftrightarrow$ Maximizes the smallest angles
 - True for 4 points :
 - Delaunay ■ Maximizes the smallest angles :
 - Proof : Suppose that the optimum triangulation is not Delaunay and prove that this leads to a contradiction.

101

101

Proof of the equivalence

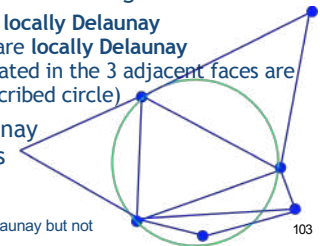
- Delaunay $\Leftrightarrow$ Maximizes the smallest angles
 - Proof for more than 4 points? (■)
 - Maximum triangulation for the lexicographical order of the smallest angles
 - > max in each convex quadrilateral (composed of 2 adjacent triangles)
 - > **Every edge is Locally Delaunay**
 - Each triangle incident to the edge **does not contain the opposite vertex** in its circumscribed circle
 - Also true for non convex quadrilaterals



102

Theorem

- Delaunay $\Leftrightarrow$ Maximizes the smallest angles (■)
 - Proof for more than 4 points?
 - Maximum triangulation for angle order
 - > All the edges are **locally Delaunay**
 - > All the triangles are **locally Delaunay** (ie. the vertices located in the 3 adjacent faces are outside the circumscribed circle)
 - Does locally Delaunay everywhere Implies globally Delaunay?



Example of a triangle locally Delaunay but not Delaunay

103

Other theorem

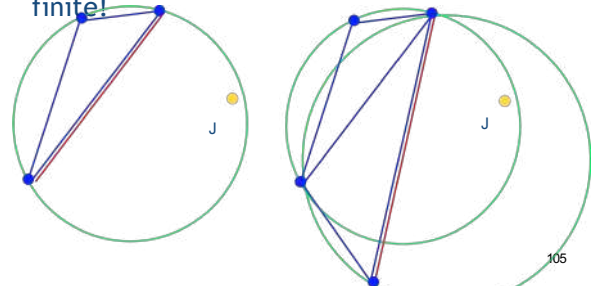
- Locally Delaunay on all triangles $\Leftrightarrow$ Delaunay triangulation (■)
- Demonstration :
 - Suppose there is a triangulation with locally Delaunay triangles, but which is not Delaunay
 - This means that the circumscribed circle of ONE triangle T has a vertex J in its interior
 - But the edge of T visible from J is incident to a triangle T' whose third vertex is outside also the circle circumscribed to T
 - J is in the circle circumscribed to T'....

104

104

Demonstration

- Progressively, we build an infinity of triangles.
- Impossible since the number of points is finite!



105

105

Transformation of a 2D triangulation

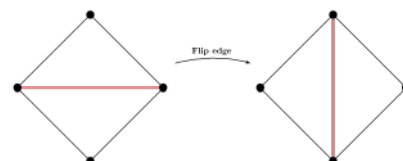
- How to improve a badly shaped triangulation?

106

106

Transformation of a 2D triangulation

- Notion of non-locally Delaunay edge
- Lawson algorithm
 - Flip non-locally Delaunay edges



107

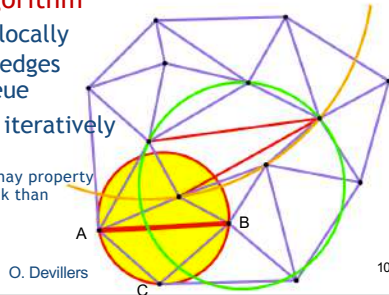
107

Transformation of a 2D triangulation into a Delaunay triangulation

- Notion of non-locally Delaunay edge

- **Lawson algorithm**

- Push non-locally Delaunay edges into a queue
- Flip them iteratively
- Note : Locally Delaunay property faster to check than Delaunay!!!!



O. Devillers

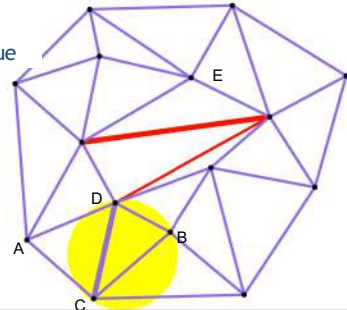
108

108

Transformation of a 2D triangulation into a Delaunay triangulation

- Flip non-locally Delaunay edges

... Using a queue

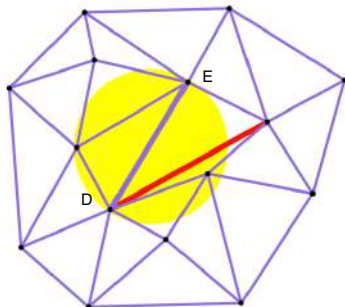


109

109

Transformation of a 2D triangulation into a Delaunay triangulation

- By non-locally Delaunay edge flipping

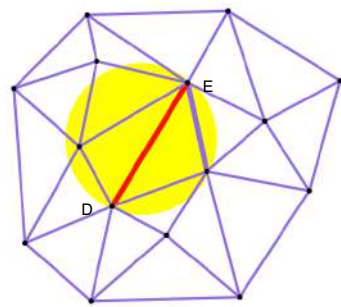


110

110

Transformation of a 2D triangulation into a Delaunay triangulation

- By non-locally Delaunay edge flipping

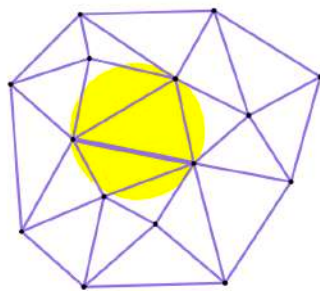


111

111

Transformation of a 2D triangulation into a Delaunay triangulation

- By non-locally Delaunay edge flipping



112

112

Transformation of a 2D triangulation into a Delaunay triangulation

- Do you think that such an algorithm should always terminate?
 - YES!
 - Think of an energy of the triangulation based on the sum of the smallest angles of all the triangles.
 - The energy decreases at each flip and the number of triangulations of a finite number of points is finite.

113

113

Note

- It is always possible to flip a non-locally Delaunay edge, because the 4 vertices of the two incident triangles are always in convex position

114

114

Properties

- Delaunay triangulation of points in « general » position is unique
 - General = No four cocyclic points
- The Delaunay triangulation of four cocyclic points is not unique.
- Possible concept of « disturbance » for determinist algorithms
 - The points are given with a speed vector, as if they were moving...

115

115

Properties

- There are interesting interpretations of the Delaunay triangulation, lifting the points into a space of higher dimension
 - What is Delaunay in 1D?
 - Delaunay is like sorting and linking each point with the next one
 - OR
 - Projecting the 1D points on the lower side of a 2D convex and computing the convex envelop of the 2D lifted points
 - Delaunay = lower envelope of points

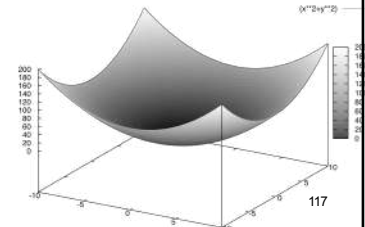
116

116

Properties

- Nice interpretation of Delaunay in the space (« space of spheres »)
 - Correspondence between the Delaunay triangulation and the lower convex envelope of the points lifted on the paraboloid

$$z = x^2 + y^2$$

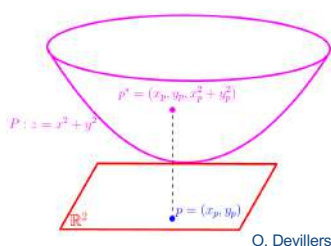


117

117

- Each 2D point x_P, y_P is lifted to $x_P, y_P, x_P^2 + y_P^2$ on the paraboloid

$$z = x^2 + y^2$$



118

118

Demonstration

- Let us consider a circle of the plan

$$(M - C)^2 = R^2$$

$$(x - x_C)^2 + (y - y_C)^2 = R^2$$

$$x^2 + y^2 - 2xx_C - 2yy_C + x_C^2 + y_C^2 - R^2 = 0$$

$$x^2 + y^2 - 2ax - 2by + c = 0$$

- Where are the points of the circle lifted on the paraboloid?



119

119

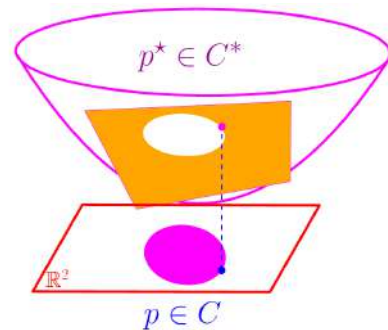
Demonstration

- Circle $x^2 + y^2 - 2ax - 2by + c = 0$
 - all the points belonging to this circle are lifted to points belonging simultaneously to the paraboloid and the plane
- $$z - 2ax - 2by + c = 0$$

120

120

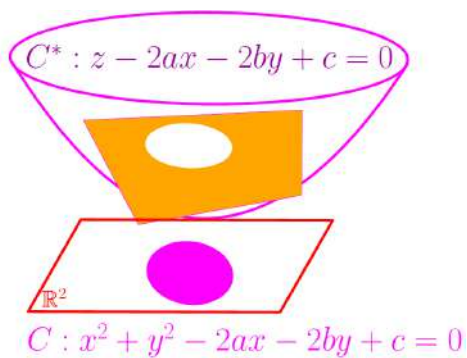
Demonstration



121

121

Demonstration

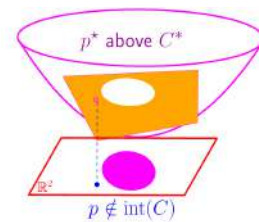


122

122

Demonstration

- What about the points outside the circle C when they are lifted?
- They are lifted above the plane C^*

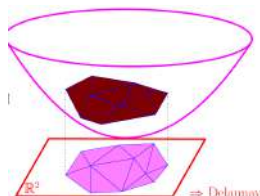


123

123

Demonstration

- If 3 points correspond to a Delaunay triangle, none of the other points can be lifted under the plane corresponding to the 3 points :
 - Correspondence between the Delaunay triangulation and the lower envelope of the points lifted to the paraboloid.



124

124

Back to Lawson's algorithm

- Transformation of a 2D triangulation into a Delaunay triangulation
 - As long as there is a non-locally Delaunay edge (ie. a non Delaunay subtriangulation of 4 points)
 - Replace the 4 points' subtriangulation by the alternative subtriangulation (flip)
- What complexity?

125

125

Back to Lawson's algorithm

- By using Delaunay interpretation in the spheres space, let's show that **an edge cannot appear more than once in the queue** (even with other incident triangles)

126

126

Back to Lawson's algorithm

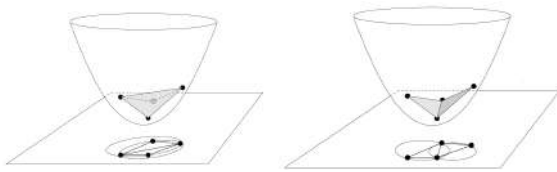
- Interpretation of the *flip* in the “space of spheres”
 - Before the *flip*, each of the two triangles incident to the edge has the 4th point in its circumscribed circle
 - The lifting of the plane of each triangle on the paraboloid is located above the 4th point.
 - Concavity of the lifted triangulation

127

127

Back to Lawson's algorithm

- Interpretation of the *flip* in the “space of spheres”



- Each *flip*, allows the lifted surface to descend locally

128

128

Back to Lawson's algorithm

- This means that a flipped edge cannot reappear a second time in the flow of the algorithm
- The number of edges that can be formed with n points is $n(n-1)/2$
- Lawson's algorithm is therefore in $O(n^2)$
- Much more efficient in practice!

129

129

Operations for triangulating and flipping edges

- Updating the Mesh data structure
 - Division of a triangle into 3 triangles (FaceSplit)
 - Division of an edge into two edges (EdgeSplit)
 - Flip of an edge (Flip)



130

130

Other use of the flip operation

- Insertion of a point P outside the convex-hull of a triangulation
 - New triangles joining P and the boundary edges that are visible from P
 - The infinite faces should be updated accordingly

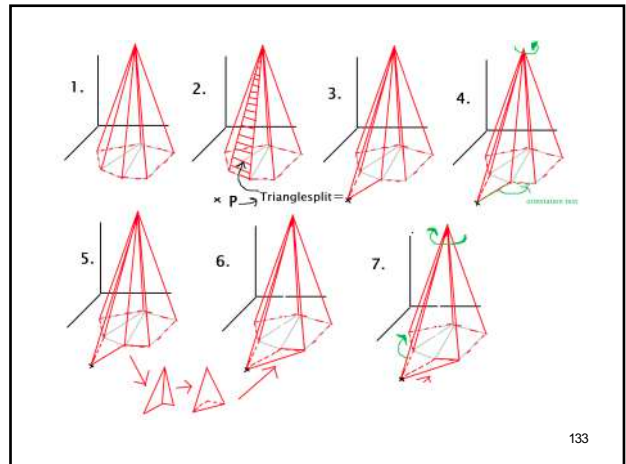
131

131

Other use of the flip operation

- Insertion of a 2D point P outside the convex-hull of a 2D triangulation
- A possible implementation using the infinite vertex and the flip operation
 - Let InfF be an infinite face incident to a visible edge of the convex-hull
 - Split InfF into 3
 - Iteratively flip the infinite edges bounding the modified area if they are incident to an other infinite face to be destroyed (starting from InfF)

132



133

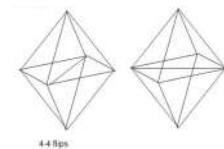
Delaunay triangulation in any dimension n

- Paving the convex-hull of the points with n-simplices whose circumscribed sphere is empty
- Warning: Lawson's algorithm is only valid in 2D, because the notion of flip is more complicated in larger dimensions

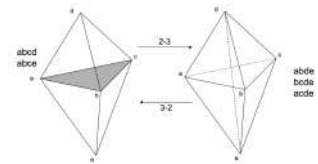
134

Flip in 3D

- 4-4 flip



- 2-3 and 3-2 flips



135

Geometric algorithms implementation

- Distinction between **exact**, **combinatorial** vs. **approximate** objects
- Input data (considered as exacts) used to construct exact non combinatorial objects
 - points x,y or x,y,z
- Construction of combinatorial objects from the exact ones
- Approximate objects should be constructed for visualisation purpose only

136

Algorithmic using predicates

- The progress of an algorithm should only depend on the sign of predicates evaluated **accurately**
 - Use of a controlled arithmetic
 - No use of inexact objects in the evaluation of predicates

137

Algorithmic using predicates

- Without these precautions, there is a risk of aberrant behaviour of a geometric algorithm
- Example: How to express the simple insertion algorithm in a 2D triangulation according to these criteria?

138

138

Algorithmic using predicates

- Algorithm of simple insertion in a 2D triangulation:
 - Using the three points orientation predicate
 - To perform inclusion tests in a triangle
 - To perform visibility tests on an edge of the convex envelope

139

139

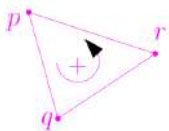
Algorithmic using predicates

- Three 2D points orientation predicate:

$$\text{orientation}(p, q, r) = \text{sign}(((q - p) \times (r - p)).Oz)$$

$$\text{orientation}(p, q, r) = \text{sign}((q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x))$$

$$= \begin{vmatrix} q_x - p_x & r_x - p_x \\ q_y - p_y & r_y - p_y \end{vmatrix}$$



$$\text{orientation}(p, q, r) = \text{sign}(\det \begin{bmatrix} 1 & p_x & p_y \\ 1 & q_x & q_y \\ 1 & r_x & r_y \end{bmatrix})$$

140

140

Algorithmic using predicates

- How to evaluate this orientation predicate?
 - In the case where the input coordinates belong to the regular grid of integers?
 - In the case where the input coordinates are rationals?

141

141

Algorithmic using predicates

- How to evaluate this orientation predicate?
 - In the case of the input coordinates belong to the regular grid of integers?
 - In the case where the input coordinates are rationals?
 - In both cases the evaluation can be carried out accurately since we benefit from an exact multiplication, addition and subtraction for these types of numbers!

142

142

Algorithmic using predicates

- How to evaluate this orientation predicate?
 - In the case where input coordinates are double?

$$\pm m 2^e \quad -1023 < e < 1024$$

$$m = 1.m_1 m_2 \dots m_{52} \quad (m_i \in \{0, 1\})$$

- The result of the arithmetic operations is rounded to the nearest double

143

143

Algorithmic using predicates

- How to evaluate this orientation predicate?
 - In the case where input coordinates are double?
 - It is only the sign of the predicates that matters
 - Is interval arithmetic enough to disambiguate the sign of a predicate?

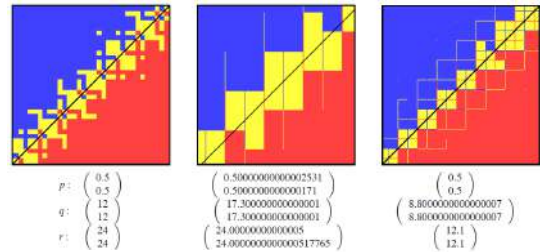
- The case where the three points are almost aligned can be error-prone

144

144

Algorithmic using predicates

- If the orientation predicate is evaluated using double arithmetic :
 - Result of orientation(p,q,r) with $p(p_x+Xu_x, p_y+Yu_y)$
 $0 \leq X, Y \leq 255 \quad u_x = u_y = 2^{-53}$



Images by S. Pion

145

Algorithmic using predicates

- If the orientation predicate is evaluated using double arithmetic :
 - There are still values for which the sign is unambiguously certified (need to control the threshold)
 - Otherwise :
 - Consider the coordinates of p, q and r as rational (with a finer precision than that usually considered) and make the exact calculation

146

146

Algorithmic using predicates

- Example : How to express Lawson algorithm using predicates?

147

147

Algorithmic using predicates

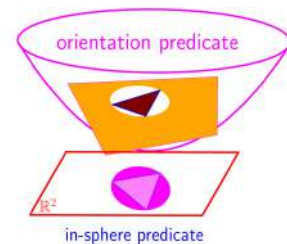
- Lawson Algorithm :
 - An AB edge should be flipped if the circle circumscribed to one of its 2 incident triangles ABC contains point D located on the other side
 - **Do not** use an inaccurate temporary object (e.g. the centre of a circumscribed circle) when evaluating a predicate

148

148

Algorithmic using predicates

- Reminder: the in-circle inclusion test can be expressed as an orientation test in a space of higher dimension (space of spheres)



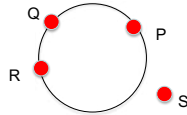
Images par O. Devillers

149

149

Algorithmic using predicates

- Predicate of inclusion of a point s in a circle circumscribed at p, q and r (orientation of the 4 points lifted on the paraboloid centered at p)
 Φ = lifting operator



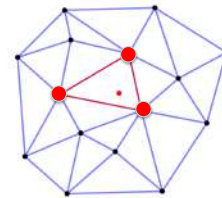
- $\text{In_circle}(p, q, r, s)$
 $= -\text{signe}(((\Phi(q) - \Phi(p)) \times (\Phi(r) - \Phi(p))) \cdot (\Phi(s) - \Phi(p)))$
 $= -\text{signe} \begin{vmatrix} q_x - p_x & r_x - p_x & s_x - p_x \\ q_y - p_y & r_y - p_y & s_y - p_y \\ (q_x - p_x)^2 + (q_y - p_y)^2 & (r_x - p_x)^2 + (r_y - p_y)^2 & (s_x - p_x)^2 + (s_y - p_y)^2 \end{vmatrix}$

150

150

Back to Delaunay triangulation

- How to modify the incremental algorithm of insertion into a simple triangulation to obtain an incremental algorithm of insertion into a Delaunay triangulation?

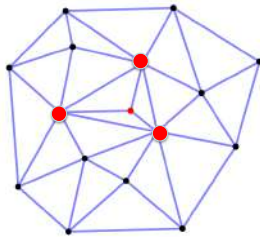


151

151

Incremental insertion into a Delaunay triangulation

- First perform a simple insertion :



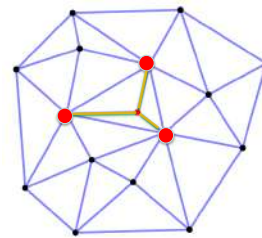
- Use Lawson flips
- Is it necessary to test all the edges?

152

152

Delaunay incremental insertion

- Let Δ be the triangle in which P was inserted
 - After the simple insertion, the triangle is star-shaped with respect to P

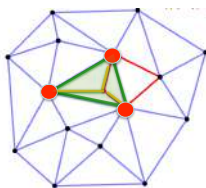


153

153

Incremental Delaunay insertion

- Let Δ be the triangle in which P was inserted
 - After the simple insertion, only the 3 edges of Δ could be candidates for flipping
 - The 3 new edges incident to P could not be flipped
 - The others have not changed their pair of incident triangles

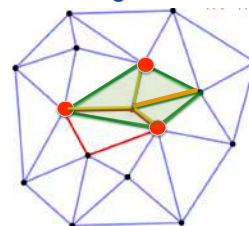


154

154

Incremental Delaunay insertion

- Each flip generates a new edge incident to P and two edges are added to the boundary of the modified area (in green)
- New green edges should be checked in turn (one of their incident triangles has been modified)

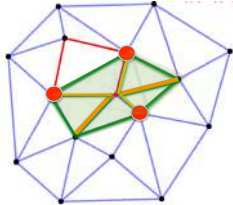


155

155

Incremental Delaunay insertion

- Each flip generates a new edge incident to P and two edges are added to the boundary of the modified area (in green)
- New green edges should be checked in turn (one of their incident triangles has been modified)

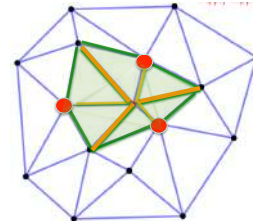


156

156

Incremental Delaunay insertion

- Each flip generates a new edge incident to P and two edges are added to the boundary of the modified area (in green)
- New green edges should be checked in turn (one of their incident triangles has been modified)



157

157

Incremental Delaunay Algorithm

- The complexity of each insertion directly corresponds to the final number i of edges incident at the new vertex.
 - Every flipped edge gave birth to an edge incident to P
->there are $i-3$ flips
 - The flipping test was checked on each edge that was effectively flipped, and also on the green boundary of the resulting modified area (i edges)

158

158

Incremental Delaunay Algorithm

- An insertion outside the convex envelope also starts as the simple insertion into a triangulation
 - Additional flips can be performed on the boundary of the modified area

159

159

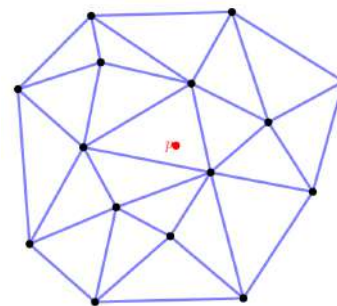
Delaunay incremental insertion (Alternative version)

- We just showed that the modified area of the triangulation is star-shaped around the inserted point P
- Alternative approach for incremental Delaunay :
 - Delete all the triangles whose circumscribed circle contains point P
 - Those triangles are said to be "in conflict" with P
 - Triangulate the conflict zone by star-shaping the conflict area around P

160

160

Incremental Delaunay Algorithm (Alternative Version)

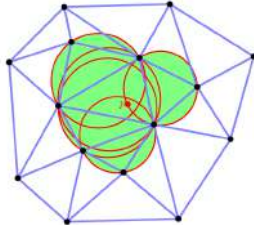


161

161

Delaunay incremental algorithm (Alternative Version)

- Determine the in-conflict triangles
 - Using breadth-first search (or depth-first search) on the adjacency graph of triangles starting from Δ

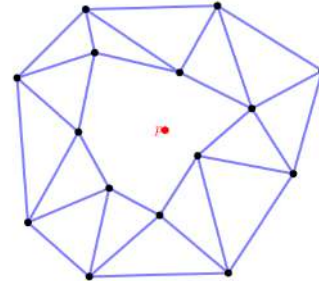


162

162

Delaunay incremental algorithm (Alternative Version)

- Conflict zone determination

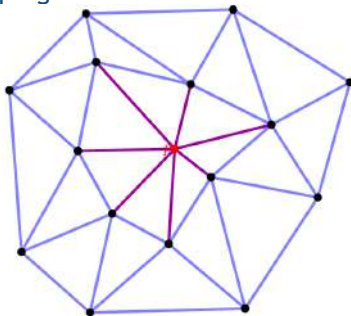


163

163

Delaunay incremental algorithm (Alternative Version)

- Star-shaping of the conflict area



164

164

Delaunay incremental algorithm (Alternative Version)

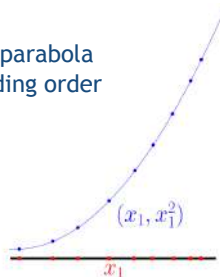
- Star-shaping of the conflict area
- Validity of this algorithm in higher dimension
 - In 2D, the **edges of the boundary** of the conflict zone get connected to the inserted point by constructing new triangles
 - In 3D, the Delaunay triangulation is composed of tetrahedrons. The **triangles of the boundary** of the conflict zone get connected to the inserted point by constructing new tetrahedrons

165

165

Incremental Delaunay Algorithm

- Complexity analysis
- Worst case :
 - Points distributed on a parabola and inserted in descending order of abscissa.

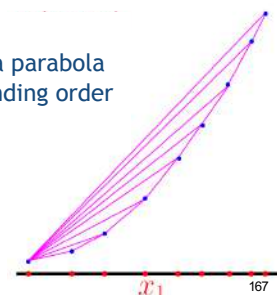


166

166

Incremental Delaunay Algorithm

- Complexity analysis
- Worst case :
 - Points distributed on a parabola and inserted in descending order of abscissa.



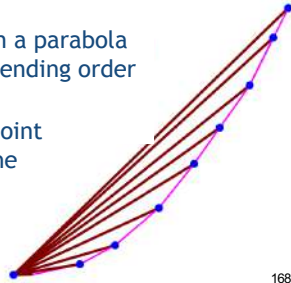
167

167

Incremental Delaunay Algorithm

- Complexity analysis
- Worst case :
 - Points distributed on a parabola and inserted in descending order of abscissa.
 - Each new inserted point conflicts with ALL the triangles

$$\Omega(n^2)$$



168

168

Incremental Delaunay Algorithm

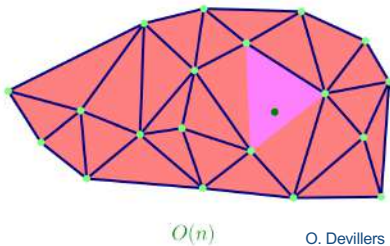
- Complexity analysis
- In average :
 - Complexity dependent on the strategy used to locate the triangle containing the point to be inserted

169

169

Location strategies

- Exhaustive search among all the triangles



O. Devillers

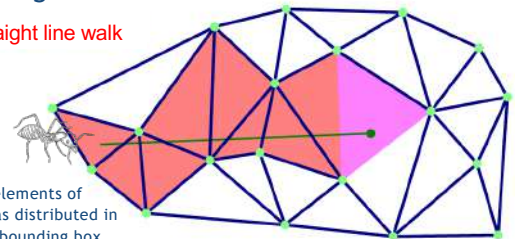
170

170

Location strategies

- Search from a first randomly selected triangle

Straight line walk



- 3n elements of areas distributed in the bounding box
- sqrt(3n) elements encountered on average on a straight line

$O(\sqrt{n})$ on average

171

171

Location strategies

- Search from a first randomly selected triangle
 - Straight walk
 - Requires a predicate of segments intersection

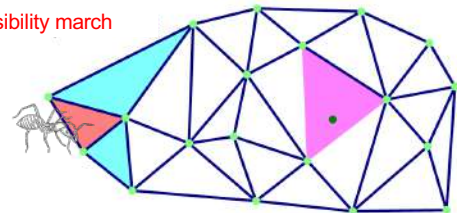
172

172

Location strategies

- Search from a first randomly selected triangle
 - Some minor deviations from the straight line walk

Visibility march



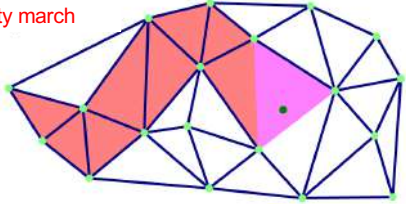
173

173

Location strategies

- Search from a first randomly selected triangle
 - Some minor deviations from the straight line walk

Visibility march



174

174

Location strategies

- Search from a first randomly selected triangle
 - Visibility walking
 - Only requires an orientation predicate to find the next triangle to walk in

175

175